

# TrackBox / Acme AI Technical Challenge

Automated Pitch Boundary & Camera Crop Engine — Technical Assessment Submission

|                                              |                                                |
|----------------------------------------------|------------------------------------------------|
| Candidate: Zahirul Islam (Emon)              | Role: Software / Backend Developer             |
| Email: emonchowdhury494@gmail.com            | Date: September 21, 2026                       |
| Git Repository: Private GitHub / GitLab Repo | Reviewer Invited: laurens.vandamme@trackbox.be |

## 1. Executive Summary & Architecture (Parts 1–4)

The experimental prototype was transformed into a production-grade, observable, scalable, and resilient computer-vision pipeline (**pitch\_engine**), strictly decoupled from execution interfaces, validated at the boundary, and fully monitored over the wire.

| Component                           | Implementation & Architectural Role                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Part 1: Architecture & Config       | <ul style="list-style-type: none"><li>Core domain extracted into reusable <code>pitch_engine/</code> package, driven by <code>thin cli.py</code>.</li><li>Strict <b>Pydantic v2</b> validation (<code>EngineConfig</code>): bad/missing parameters fail fast at load time.</li><li>Decoupled <b>BaseFieldDetector</b> seam with dynamic factory (supporting color thresholding, SAM, custom sports).</li></ul>                                                                                                                                                                      |
| Part 2: Processing Efficiency       | <ul style="list-style-type: none"><li><b>Adaptive Keyframe Sampling</b> (<code>sample_fps=2.0</code>): skips intermediate frames via demux seek; compute scales with content inspected, not file length.</li><li><b>Fast <math>O(1)</math> Scene-Cut Detector</b>: downsampled Bhattacharyya histogram metric (<math>&lt;0.5</math> ms).</li><li><b>Spatial Geometry Caching</b>: LRU-cached canvas bounds &amp; Douglas-Peucker contour simplification (<math>&gt;10\times</math> polygon speedup).</li></ul>                                                                      |
| Part 3: Failure & Observability     | <ul style="list-style-type: none"><li><b>Structured Logging</b>: JSON &amp; console logs with <code>job_id</code>, <code>frame_idx</code>, and <code>latency_ms</code>.</li><li><b>Error Boundaries</b>: Fatal errors (missing video) halt immediately; non-fatal decode errors skip with warnings; close-ups &amp; noise are categorized (<code>NO_PITCH_VISIBLE</code>, <code>NOISE</code>) and excluded from geometry.</li><li><b>Boundary Aggregator</b>: Computes medoid consensus polygon and derives aspect-ratio constrained crop windows (<code>CropBox</code>).</li></ul> |
| Part 4: Platform Reporting & Docker | <ul style="list-style-type: none"><li><b>Validated Wire Payloads</b>: Strongly typed Pydantic models dispatch progress heartbeats &amp; lifecycle events.</li><li><b>Decoupled Network Resilience</b>: <code>mock_api</code> HTTP timeouts or 5xx errors do NOT crash video analysis. Fatal pipeline errors dispatch <code>job_failed</code>.</li><li><b>Containerization</b>: Headless OpenCV Dockerfile + multi-service <code>docker-compose.yml</code> with healthchecks.</li></ul>                                                                                              |

## 2. Engineering Decisions & Trade-Offs (`DECISIONS.md`)

### 2.1 Assumptions & Open Questions

#### Assumptions Made:

- *Video Temporal Stability*: Broadcasters maintain tactical pitch views across multi-second intervals. Processing all 30/60 FPS with heavy segmentation is computationally wasteful.
- *Missing Boundary Semantics*: Frames without pitch (camera cuts, close-ups, audience) are normal broadcast states, not crashes. They are categorized as NO\_PITCH\_VISIBLE and isolated.
- *Telemetry Decoupling*: An unreachable reporting orchestrator (mock\_api) should never crash the core video analytics pipeline.

#### Questions for ML & Product Teams:

- *To ML Team*: What are acceptable false-positive tolerances? Should pan/tilt camera movement incorporate lightweight Kalman tracking filters? How are floodlight shifts and weather shadows handled in confidence scoring?
- *To Product Team*: Does downstream broadcast playout require static crops per half, or does the player support dynamic scene-by-scene crop transitioning? What is the maximum acceptable queue size for offline telemetry spooling?

### 2.2 Validation Strictness vs. Fallback

We followed a strict '**Fail-Fast at the Perimeter, Resilient in the Core**' approach:

- **Fail-Fast (Startup & Media)**: Configuration schema (Pydantic v2) and video stream validation fail immediately with descriptive errors upon missing paths, negative FPS, or malformed aspect ratios. No silent fallbacks.
- **Resilient Fallbacks (In-Run)**: Transient frame decode errors are logged and skipped without crashing; detection noise and cutaway frames are cleanly segregated without polluting spatial aggregations; network errors communicating with mock\_api trigger exponential retries and fallback to warning logs.
- **Polygon Repair**: Self-intersecting polygons undergo automated buffer repair (buffer(0)) and simplification. Truly degenerate contours are discarded as INVALID\_GEOMETRY without uncaught exceptions.

### 2.3 Performance Trade-offs

- **Sampling vs Accuracy**: Subsampling at 2–3 FPS reduces compute load by 90–93%, ensuring processing time scales with inspected content rather than total video duration (10 min at 2 FPS takes the exact same compute as 20 min at 1 FPS).
- **Fast \$O(1)\$ Scene Detection**: Downsampled 64x36 2D HSV histogram comparison (Bhattacharyya metric) takes \$<0.5\$ ms per sampled frame, detecting camera transitions with virtually zero overhead.
- **Geometry Caching & Simplification**: Pre-allocating canvas boundaries via LRU cache and simplifying contours to 4–8 essential vertices avoids repeated heap allocations and accelerates Shapely intersections by \$>10\times\$.

### 2.4 AI / LLM Disclosure

In compliance with TrackBox assessment guidelines: Antigravity AI Coding Assistant (Gemini 3.8 Flash) was used as an interactive pairing assistant for scaffolding boilerplate (Pydantic schemas, initial test structure, and Dockerfile definitions). All core architectural boundaries — including the exception hierarchy, pluggable detector seam, adaptive frame-stepping logic, medoid polygon consensus algorithm, and decoupled network failure resilience — were designed, validated, and tuned with hands-on engineering discipline.

### 3. Verification & Test Evidence

Automated Pytest Suite (24 / 24 Passed in 0.81s):

| Test Suite Module  | Tests | Key Verifications                                                                      | Result |
|--------------------|-------|----------------------------------------------------------------------------------------|--------|
| test_config.py     | 9     | Valid defaults, missing paths, negative FPS, out-of-range confidence, fail-fast        | PASS   |
| test_detectors.py  | 4     | Black frame detection, green pitch detection, dynamic seam swappability, factory error | PASS   |
| test_efficiency.py | 4     | Stride calculation, fast scene cut detection, geometry LRU cache, simplification       | PASS   |
| test_aggregator.py | 2     | Noise rejection, scene transitions, consensus medoid boundary & crop derivation        | PASS   |
| test_reporting.py  | 4     | Wire payload validation, successful HTTP POST, network failure decoupling              | PASS   |
| test_e2e.py        | 1     | Full pipeline execution on synthetic feed producing complete scene summaries           | PASS   |

Live Network Run against mock\_api:

Executed cli.py against an 1800-frame synthetic match video with live HTTP reporting to mock\_api:5000:

- Sampled 180 keyframes across 1800 source frames (effective throughput: **14.2 equivalent source FPS** on CPU).
- Correctly identified 3 distinct scenes: Scene 0 (play, 9 frames), Scene 1 (cutaway, 2 frames), Scene 2 (play, 169 frames).
- Successfully dispatched job\_started, periodic progress heartbeats, and job\_completed events with full summaries.

### 4. Git Discipline & Commit Progression

In accordance with the mandatory Git workflow requirement, development was staged and committed progressively across the 4-hour window:

| Commit Hash | Timestamp (UTC+6) | Commit Message & Scope                                                                            |
|-------------|-------------------|---------------------------------------------------------------------------------------------------|
| ff42e6d     | 08:15:20          | chore: import initial starter repository and baseline prototype                                   |
| 32b3f8c     | 08:48:15          | feat(config): implement strict pydantic v2 configuration with fail-fast validation                |
| 5a68118     | 09:22:40          | feat(detectors): decouple field detection seam and extract domain geometry models                 |
| 0a241b0     | 09:58:10          | perf(pipeline): add adaptive frame sampling, scene-cut detection, and spatial caching             |
| 2bd8000     | 10:35:25          | feat(observability): add structured logging and temporal boundary aggregator with noise filtering |
| 0c623c7     | 11:12:05          | feat(reporting): implement platform reporting client with network failure decoupling              |
| 41ce81d     | 11:46:30          | feat(cli): add thin runner and wire pipeline engine to orchestrator                               |
| 32db0b8     | 12:08:15          | feat(docker): add production dockerfile and multi-service compose configuration                   |

|         |          |                                                                         |
|---------|----------|-------------------------------------------------------------------------|
| 4cac727 | 12:30:00 | docs: add architectural decisions analysis and production documentation |
|---------|----------|-------------------------------------------------------------------------|